TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

# Part 4

## The editor: from FBX files to a vehicle that spawns

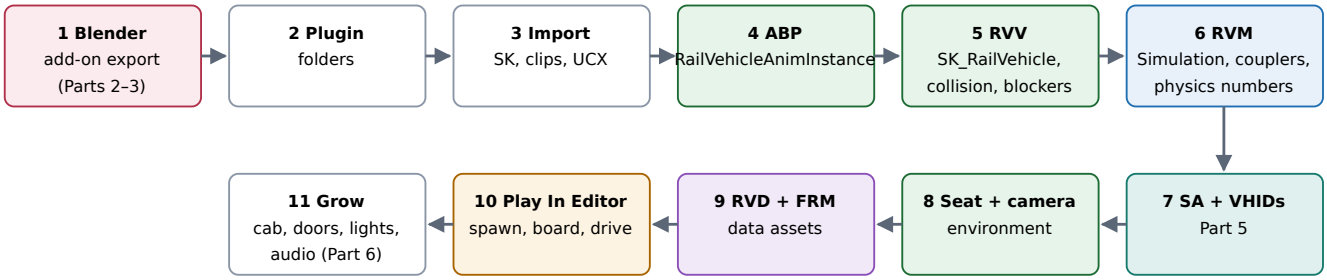
The Blender side is done; everything from here happens in the TSW PC Editor. This part is the build order for a minimum locomotive: plugin, import, Animation Blueprint, RVV, RVM, physics numbers, seat and camera, RVD and formation, Play In Editor. Controls and the SimuGraph are Part 5; do them right after §7 of this part. Each step is testable on its own.

### In this part


1. Build order
2. Plugin and folders
3. Import the FBX set
4. The Animation Blueprints
5. The RVV Blueprint
6. The RVM Blueprint and the physics numbers
7. Environment, seat and camera
8. RVD and formation
9. Play In Editor
10. Growth path


Steps follow the community wiki's “Setting up a Rail Vehicle” guide where it exists and otherwise the structure read out of Thomas, the Class 47 and the 12-ton van (verified). The editor-side Python helper (Part 7) automates §2, §3 and §8 but was written here without running the editor (check).

# 1 · Build order




Do not model the cab, lights or audio until step 10 works. A box on two axles that drives is the first milestone.

**Figure 4.1 · Eleven steps.** Steps 2, 3 and 9 are scripted by the editor helper; 4–8 are clicks in the editor with the build sheet open.

# 2 · Plugin and folders

> **Before the first launch**
>
> Run the game itself once before starting the editor; the editor can refuse to start (or show a dialog to confirm) until the game has initialised its files. The editor opens on the **frontend level**, the menu scene: nothing is built there. Open a route map (the Training Center map is a good test bed) and, before any Blueprint exists, drag the imported skeletal mesh into it to check scale and materials. community

1. Edit ▸ Plugins ▸ **New Plugin** ▸ Content Only ▸ name `MyLoco`. Never put files in `/Game/` or in a DLC folder.
2. Content Browser ▸ View Options ▸ **Show Plugin Content**; open *MyLoco Content*.
3. Create the folders from Part 0: `Data/RVD`, `Simulation`, `View/Models/Exterior`, `View/Models/Exterior/Animation`, `View/Models/Cab`, `View/Models/Collision`, `View/Materials`, `Audio/RVA`, `Audio/Cues`. (The helper script makes them.)

# 3 · Import the FBX set

| File | Into | Import settings |
|---|---|---|
| `SK_MyLoco_Ext.fbx` | View/Models/Exterior | Skeletal Mesh ✓, Skeleton: none (creates `SK_MyLoco_Ext_Skeleton`), Import Animations ✗, Use T0 As Ref Pose ✓, Create Physics Asset ✓, Materials ✓ the first time. Check the skeleton's socket list afterwards. check |
| `SK_MyLoco_Cab.fbx` | View/Models/Cab | Same, but **Skeleton = SK_MyLoco_Ext_Skeleton**. |

| File | Into | Import settings |
|---|---|---|
| `Animation/*.fbx` | View/Models/Exterior/Animation | Import Mesh ✗, Import Animations ✓, Skeleton = the exterior skeleton, Animation Length = Exported Time. Each file becomes one AnimSequence named after its take (= the clip). |
| `UCX_MyLoco_Collision.fbx` | View/Models/Collision | Static Mesh, Combine Meshes ✓, Auto Generate Collision ✗; then in the asset set Collision Complexity = *Use Complex Collision As Simple*. check |
| Textures | View/Materials | Normal maps: Flip Green Channel if baked in Blender. ORM: sRGB ✗. |

**What to look at after the import**

Open the skeletal mesh. The bone tree should read `Root ▸ Body_Pivot ▸ …` with `axle1..n`, `bogieN` and the door/lever bones; if an extra top bone named `Armature` appears, it is harmless for this workflow. The Skeleton tab lists the sockets: `Wheel_L1` …, `DriverSeat`, `Coupler_F/B`, your levers and audio points. Scrub a clip in the animation editor: the door should swing about its hinge. check

# 4 · The Animation Blueprints

1. Right-click the skeleton ▸ Create ▸ **Anim Blueprint**, parent class `RailVehicleAnimInstance`, name `ABP_MyLoco_Ext`.
2. **Class defaults** from the build sheet: `Axles.Entries` = [axle1, axle2, …]; `Bogies.Entries` = [bogie1, bogie2] (if any); `BufferBones` = [buffer_fl, …]; `VirtualHIDSequenceEntries`: one row per door / exterior VHID: Component "CabDoor_L" → Sequence `Door_L1`, etc.
3. **AnimGraph** (Figure 3.3): Mesh Ref Pose → Apply Additive (additive input = Sequence Evaluator `Wheels_FullRotation`, explicit time bound to a float `AxleRotationAlpha`) → VehicleAnimGroupEvaluation → Component To Local → Output Pose.
4. **Variable** of the simulation parameter reference type (NodeReference + ParameterReference), set to component `Simulation`, node `Axle1`, parameter `WheelRotation_DEG`. Event graph on update: `AxleRotationAlpha = value / 360`. If the getter is not obvious, bind the variable through the Details panel's property binding as Thomas's ABP does. check
5. `ABP_MyLoco_Cab`: same parent, no wheel evaluator; `VirtualHIDSequenceEntries` = the lever clips (ThrottleControl → Throttle, ReverserControl → Reverser, BrakeControl → Brake).

# 5 · The RVV Blueprint

1. In `View`: Blueprint Class ▸ All Classes ▸ **RailVehicleView** ▸ `RVV_MyLoco`.
2. Add a Skeletal Mesh component named **`SK_RailVehicle`** (Thomas's name; the streamer's wagon called it `Body` and worked, so the name is a convention rather than a rule community): mesh `SK_MyLoco_Ext`, Animation Mode = Blueprint, class `ABP_MyLoco_Ext`.

3. Add `SK_Cab` (skeletal mesh, `ABP_MyLoco_Cab`) once the cab exists.
4. Add a Static Mesh component `SM_Collision` with the UCX mesh: hidden in game, collision enabled.
5. Add two Box components `Blocker_F` / `Blocker_B` at the ends of the footplate so the player cannot walk off.
6. Add a `ComponentLODManagementComponent`.
7. Leave *Reference Actor* empty for now; the RVM does not exist yet. Save.

# 6 · The RVM Blueprint and the physics numbers

1. In `Simulation`: Blueprint Class ▸ All Classes ▸ **RailVehicle** ▸ `RVM_MyLoco`.
2. Class Defaults ▸ TSW Extensions ▸ **Reference Actor** = `RVV_MyLoco`. Then open `RVV_MyLoco` and set *its* Reference Actor to `RVM_MyLoco`. (At runtime these are `ModelChildActorComponent.view_actor_asset` and `RVV.ModelActor`.) **verified**
3. Add Component ▸ **SimulationComponent**, name `Simulation`. Its SimuGraph asset comes in Part 5. Thomas runs it with `bEnableParallelism` on; the streamer switches parallelism **off** on custom vehicles, having had problems with it. **community**
4. Add Component ▸ **InteractionEnvironmentComponent**, name `DriverEnvironment`, role DRIVER, side FRONT.
5. Add Component ▸ **SeatManagerComponent**, name `SeatManager`.
6. Add two **CouplerComponents**: `Coupler_F` (side FRONT) and `Coupler_B` (BACK). Set `coupler_id` (to couple to Dovetail stock read the id off a running vehicle, Part 6 §6; Thomas uses `ThomasAndFriends_Hook`; the streamer's SA3 stock uses its own id, and only matching ids couple), pivot and receiving points from the build sheet (X along, Y up, metres). **Pivot point** = where the coupler leaves the body (draft gear / coupler root); **receiving point** = where two couplers meet (the knuckle lock); heights are similar across stock, X is yours to measure. For knuckle or SA3 couplers tick `can_auto_couple`; buffer springiness off on stock without buffers; slack range about ±0.01 m; make the rear one by duplicating the front and negating X. **community**
7. Select the inherited **RailVehiclePhysicsComponent** and type the build sheet's numbers:

| Property | Build sheet key | Boilerplate / Thomas |
|---|---|---|
| `Node.Width / Height / Length` (m) | `physics.Width_m` … | 2.4 / 2.9 / 6.0 · Thomas 2.42 / 2.90 / 6.54. Length is measured over the coupler faces, not the body; the build sheet includes the coupling empties for that reason. |
| `Node.Mass` (kg) | `physics.Mass_kg` (fill in) | 40 000 · Thomas 50 000 |

| Property | Build sheet key | Boilerplate / Thomas |
|---|---|---|
| `Node.PhysicsBodyOffset`, `CentreOfMassOffset` (x, z m) | `PhysicsBodyOffset_xz_m`, `CentreOfMassOffset_xz_m` | (0, 1.5), (0, 1.2) · Thomas (−0.04, 2.13), (0, 1.5). If the vehicle floats above or sinks into the rails, the body offset Z must be half the height (the streamer's fix). **community** |
| `Node.CreepCoefficient`, drag front/back/side | | 0.002, 1.0 each |
| `Node.bHasFront/BackBuffers`, sizes, offsets, stiffness | `buffers_m` (negate Y for UE) | size (0.25, 0.35, 0.35), stiffness 3000 |
| `SimplePhysics`: bPowered, MaxTractiveEffort, MaximumBrakingEffort, MaxSpeed, curve | from your TSC sim blueprint | true, 150 000 N, 120 000 N, 20 m/s, Dovetail's shared curve in `/Game/Core/RailVehicles/RVM/` |
| `StopShortDistance`, MaxComfortableAcceleration/Deceleration | | 1.8 m, 0.45 m/s² |

Then do Part 5 (the SimuGraph and the three controls) before the seat, because the controls need the environment and the HUD needs the controls.

# 7 · Environment, seat and camera

On the RVV: **verified** field names

- **SeatComponent** `DriverSeat`: seat_role ENGINEER, seat_side FRONT, character_seat_socket `DriverSeat`, initial_camera_component `DriverCamera`; hit box and entrance box extents around the seat.
- **InteractionEnvironmentCameraComponent** `DriverCamera`: camera_name “Driver's Seat”, camera_location INTERIOR, interaction_environment_component `DriverEnvironment`, FOV 90 (min 50), freedom of rotation pitch 135 / yaw 360.
- Two **CameraTransitionComponents** `StepUp_L` / `StepDown_L` at the cab door so the on-foot player can board; `camera_transition_max_speed` stops boarding a moving train.
- Later: `HeadoutLeft/Right` cameras (EXTERIOR), a second seat and environment for the second man.

# 8 · RVD and formation

1. In `Data/RVD`: right-click ▸ Miscellaneous ▸ Data Asset ▸ **RailVehicleDefinition** ▸ `RVD_MyLoco`.
2. ModelActorAsset = `RVM_MyLoco`, ViewActorAsset = `RVV_MyLoco`, VehicleCategory Locomotive, MaximumSpeed (m/s), FriendlyName, bIsDrivable ✓, bIsSelectable ✓, bIsAvailableForFormationEditor ✓.
3. Tick **bRegenBounds**, then ConnectivityData ▸ **bRecalculateData** so the coupler ids are copied from the RVM.

4. **Region:** the game's menus filter by the regions in its own table ( `CountryTableRow` ); a custom region string is not filterable, so pick the nearest existing one (the streamer chose Germany for a Ukrainian wagon). For a wagon: category Wagon, PoweredAxleCount 0, service types as wanted, a thumbnail later (Part 6 §8). community

5. Next to it: Data Asset ▸ **TrainFormation** ▸ `FRM_MyLoco` with one Train Entry pointing at `RVD_MyLoco` .

# 9 · Play In Editor

1. Open a route you own (LeedsManchesterMap works), press **Play**, and spawn `FRM_MyLoco` the same way Dovetail's stock is spawned (free-roam spawn menu or the `ts2.rtt` console commands the community God Mode tool wraps). check The route the streamer used instead: open the map's **timetable** tool, add a train, choose your formation, *Pick Location* on a track, rebuild the timetable, then Play ▸ *current camera location*. Two formations placed touching each other coupled by themselves. community

2. Walk up, click the step, click the seat. The HUD should show your levers. Reverser forward, brake off, throttle up.

3. If it does not move: the sim runs without the ABP, so the ABP is not the problem; confirm the RVM graph writes to the exact node names; look at the live SimuGraph values with the Python snippet in Part 6 §6.

4. If it moves but the wheels do not turn: the ABP binding (node name, parameter name), the Axles list, or the ABP's model actor reference and the per-entry sim node references (Part 3 §8).

5. If it jumps or twists when hauled: two bogie nodes in the SimuGraph share the same offset (Part 5 §2).

6. Save everything; a crash in PIE loses unsaved Blueprints.

# 10 · Growth path

| Feature | What to add |
|---|---|
| Cab interior | `SK_Cab` + `ABP_MyLoco_Cab` with VHID entries for each lever clip; move the hitboxes onto the cab's lever sockets. |
| Doors | RVM `RailVehicleDoorComponent` CabDoor_L/R, RVV hitboxes, door clips in the exterior ABP, door cue in the RVA. |
| Lights | RVM PushButton Headlights + VHID preset; RVV LinearElectricLightComponent + SpotLight; bind the VHID event in the RVV graph. Part 6. |
| Audio | `RVA_MyLoco` (parent RailVehicleAudio) in the RVD's AdditionalViewActorAssets; running loop + horn; speed thresholds. Part 6. |
| Realistic braking | Replace BrakeInput with DirectBrakeControl + compressor + reservoirs + hoses, plus AirHoseComponents on the RVM. Part 5. |
| Second cab | Duplicate environment, seat, camera and every control with `_B` ; set bIsSlaveLinked on the rear levers. |

| Feature | What to add |
| --- | --- |
| Safety systems, wipers, sanders, numbering, crew | One more VHID pair each plus a SimuGraph block or a view component (WiperComponent, NumberingComponent + NumberBinding, CharacterCollection on the RVD). |